

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,  
# THEN FEEL FREE TO DELETE THIS CELL.  
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON  
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR  
# NOTEBOOK.  
  
fiktus_mod_pytorch_default_1_path = kagglehub.model_download('fiktus/mod/PyTorch/default/1')  
  
print('Data source import complete.')
```

Школа глубокого обучения ФПМИ МФТИ

Домашнее задание. Классификация изображений



- ✓ Домашнее задание. Классификация изображений

Сегодня вам предстоит помочь телекомпании FOX в обработке их контента. Как вы знаете, сериал "Симпсоны" идет на телеэкранах более 25 лет, и за это время скопилось очень много видеоматериала. Персонажи менялись вместе с изменяющимися графическими технологиями, и Гомер Симпсон-2018 не очень похож на Гомера Симпсона-1989. В этом задании вам необходимо классифицировать персонажей, проживающих в Спрингфилде. Думаю, нет смысла представлять каждого из них в отдельности.

В нашем тесте будет 991 картинка, для которых вам будет необходимо предсказать класс.

kaggle nickname - Semen

- ✧ kaggle team - Семён_Фурсов_1171677680

для удобства результат написал в самом низу

- ✓ Шаг 1. Установка зависимостей

- Установим необходимые библиотеки и проверим доступность CUDA

```
# we will verify that GPU is enabled for this notebook
# following should print: CUDA is available! Training on GPU ...
#
# if it prints otherwise, then you need to enable GPU:
# from Menu > Runtime > Change Runtime Type > Hardware Accelerator > GPU
```

```
CUDA is available! Training on GPU ...
```

```
!nvidia-smi
```

Thu Apr 30 17:42:25 2026

NVIDIA-SMI 580.105.08				Driver Version: 580.105.08		CUDA Version: 13.0	
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.
							MIG M.
0	Tesla T4		Off	00000000:00:04:0	Off		0
N/A	39C	P8	10W / 70W	3MiB / 15360MiB		0%	Default N/A
1	Tesla T4		Off	00000000:00:05:0	Off		0
N/A	46C	P8	11W / 70W	3MiB / 15360MiB		0%	Default N/A

```
+-----+
| Processes: |
| GPU  GI  CI           PID  Type  Process name                      GPU Memory |
|      ID  ID                                   Usage                        |
|=====|
| No running processes found |
+-----+
```

```
import pickle
import numpy as np
from skimage import io

from tqdm import tqdm, tqdm_notebook
from PIL import Image
from pathlib import Path

from torchvision import transforms
from torchvision.transforms import v2

import torchsummary

from multiprocessing.pool import ThreadPool
from sklearn.preprocessing import LabelEncoder
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn

from matplotlib import colors, pyplot as plt
%matplotlib inline

# в sklearn не все гладко, чтобы в colab удобно выводить картинки
# мы будем игнорировать warnings
import warnings
warnings.filterwarnings(action='ignore', category=DeprecationWarning)
```

- ✓ Проверим версии Python и библиотек

Посмотрим, какие версии установлены:

```
!python --version
from importlib.metadata import version
for b in ['torch', 'torchvision', 'tqdm', 'matplotlib', 'seaborn', 'Pillow']:
    print(b + ' version: ' + version(b))
```

```
Python 3.12.12
torch version: 2.10.0+cu128
torchvision version: 0.25.0+cu128
tqdm version: 4.67.3
matplotlib version: 3.10.0
seaborn version: 0.13.2
Pillow version: 11.3.0
```

Сверим с версиями питона и библиотек, на которых работал эталонный ноутбук:

- Python 3.12.12
- torch version: 2.8.0+cu126
- torchvision version: 0.23.0+cu126
- tqdm version: 4.67.1
- matplotlib version: 3.10.0
- seaborn version: 0.13.2
- Pillow version: 11.3.0

- ✓ Определим константы, которые будем использовать в по ходу ноутбука

```
# разные режимы датасета
DATA_MODES = ['train', 'val', 'test']

# работаем на видеокарте
DEVICE = torch.device("cuda")

#определим директории с тренировочными и тестовыми файлами
TRAIN_DIR = Path('/kaggle/working/train') #Path('./data/train/')
TEST_DIR = Path('/kaggle/working/testset') #Path('./data/testset')

# параметры нормировки изображений по трем каналам перед подачей в модель
NORMALIZE_MEAN = [0.485, 0.456, 0.406]
NORMALIZE_STD = [0.229, 0.224, 0.225]

# все изображения будут масштабированы к размеру 224x224 px
RESCALE_SIZE = [224, 224]
```

- Шаг 2. Загрузка и обработка данных

- ✓ Скачаем изображения по ссылке

!gdown 1RxBQiZgRAfio2tWhEE7lzZ6IaJzLheH1

```
Downloading...  
From (original): https://drive.google.com/uc?id=1Rx80jZgRAfio2tWhEE7lZz6IaJzLheH1  
From (redirected): https://drive.google.com/uc?id=1Rx80jZgRAfio2tWhEE7lZz6IaJzLheH1&confirm=t&uuid=389e72e0-d6c6-46aa-a9c2-cd2f3e881cd3  
To: /kaggle/working/journey-springfield.zip  
100%|███████████████████████| 546M/546M [00:04<00:00, 113MB/s]
```

```
!unzip -q /kaggle/working/journey-springfield.zip
```

Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

Посмотрите на структуру файлов в папках train и testset.

В train лежат данные, которые мы будем использовать для обучения модели. Изображения персонажей разложены по папкам, которые названы по именам персонажей. Названия папок мы в дальнейшем будем использовать в качестве текстовых меток классов.

В testset находятся изображения, для которых вам надо будет сделать прогноз наиболее вероятного класса.

Для обращения к файлам сформируем списки имен файлов обучающей+валидационной и тестовой выборки. Это полные имена, включающие путь к файлам.

```
train_val_files = sorted(list(TRAIN_DIR.rglob('*.*jpg')))  
test_files = sorted(list(TEST_DIR.rglob('*.*jpg')))
```

```
train_val_files
```

Показать скрытые выходные данные

Кодировать имена персонажей в числовые метки класса и обратно будем при помощи `LabelEncoder`.

Для `train` выборки сформируем список текстовых меток всех изображений - имя родительской директории, которая одновременно является и именем персонажа. Зададим числовые метки классов нашего энкодера при помощи метода `fit`.

Далее будем применять метод `transform` для преобразования текстовых меток в числовые, и метод `inverse_transform` для преобразования числовых меток в текстовые.

```
label_encoder = LabelEncoder()

train_val_labels = [path.parent.name for path in train_val_files]

label_encoder.fit(train_val_labels)
```

▼ LabelEncoder ⓘ ?

LabelEncoder()

Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

Разделим `train` выборку на обучающую и валидационнную части. Для того, чтобы персонажи были пропорционально представлены в обучающей и валидационнной подвыборках, применим стратификацию по меткам класса.

```
from sklearn.model_selection import train_test_split

train_files, val_files = train_test_split(train_val_files, test_size=0.25, \
                                         stratify=train_val_labels)
```

Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

▼ Создадим Datasets и Dataloaders

Полезная статья, которой мы вдохновились при создании класса `SimpsonsDataset`: https://jhui.github.io/2018/02/09/PyTorch-Data-loading-preprocess_torchvision/

У нас есть набор файлов-картинок, которые требуется преобразовать в тензоры, и добавить к ним числовые метки классов. Реализуем это с помощью созданного нами класса `SimpsonsDataset`, который из списка файлов создаст необходимый нам `Dataset`.

Будем действовать согласно инструкции из документации (https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html#creating-a-custom-dataset-for-your-files):

- наш класс будет наследовать от класса `Dataset`
- мы переопределим методы `__init__`, `__len__` и `__getitem__`, а также для удобства добавим несколько методов для загрузки и преобразования изображений.

Важно разобраться, что делает метод `self.transform_images_to_tensors()`.

`Compose` объединяет последовательность следующих преобразований:

- `PILToTensor` конвертирует `PIL Image` в тензор с параметрами в диапазоне `[0, 255]` (как все пиксели в исходном изображении)
- `ToDtype` преобразует тензор в `FloatTensor` размера $(C \times H \times W)$ со значениями пикселей в диапазоне `[0, 1]`
- затем `Normalize` производится масштабирование: $\text{input} = \frac{\text{input} - \text{mean}}{\text{std}}$, где константы `mean` и `std` - средние и дисперсии по каналам в датасете `ImageNet`
- наконец, `Resize` преобразует картинку к размеру 224×224 (в описании датасета указано, что картинки разного размера, так как брались напрямую с видео, поэтому следует привести их к одному размеру).

**

Сейчас аугментация не изображений не производится, поэтому для обучающих и валидационных/тестовых изображений производится одинаковая трансформация. В дальнейшем, если вы захотите добавить аугментацию, вы можете сделать это, например, модифицировав метод `transform_images_to_tensors`. Подробнее про трансформацию изображений вы можете почитать в документации: <https://docs.pytorch.org/vision/main/transforms.html>

**

```
class SimpsonsDataset(Dataset):
    def __init__(self, files, label_encoder, mode):
        super().__init__()
        # список файлов для загрузки
        self.files = sorted(files)
        # режим работы
        self.mode = mode
        if self.mode not in DATA_MODES:
            print(f"{self.mode} is not correct; correct modes: {DATA_MODES}")
            raise NameError

        self.label_encoder = label_encoder
        self.len_ = len(self.files)

    def __len__(self):
        return self.len_ # сейчас self.__len__() возвращает количество картинок, подаваемых на вход.
        # Если вы решите перевзвесить размеры категорий внутри класса -
        # не забудьте изменить вывод self.__len__()

        #oversampling (дублирование редкого класса) (чтобы редкие категории встречались чаще)

    def __getitem__(self, index):
        x = self.load_image(self.files[index])
        x = self.transform_images_to_tensors(x)

        if self.mode == 'test':
            return x
        else:
            path = self.files[index]
            y = self.label_encoder.transform([path.parent.name,]).item()
            return x, y

# принимает путь к файлу изображения и возвращает само изображение
def load_image(self, file):
    image = Image.open(file)
    image.load()
    return image

# преобразует изображение в тензор
```

```
def transform_images_to_tensors(self, image):
    if self.mode == 'train':

        #встроена аугментация
        transform = v2.Compose([
            v2.Resize(RESCALE_SIZE),
            v2.ColorJitter(brightness = 0.1, contrast = 0.1, saturation = 0.1), # изменить на 0.1 или ещё больше упростить
            v2.RandomHorizontalFlip(0.5),
            v2.RandomRotation(degrees=10),
            v2.PILToTensor(),
            v2.ToDtype(torch.float32, scale=True),
            v2.Normalize(NORMALIZE_MEAN, NORMALIZE_STD),
            #v2.RandomErasing(p=0.1),#области / попробовать убрать, упростить остальные параметры.

        ])
    else:
        transform = v2.Compose([
            v2.Resize(RESCALE_SIZE),
            v2.PILToTensor(),
            v2.ToDtype(torch.float32, scale=True),
            v2.Normalize(NORMALIZE_MEAN, NORMALIZE_STD),

        ])

    tensor_transformed = transform(image)
    return(tensor_transformed)

train_dataset = SimpsonsDataset(train_files, label_encoder = label_encoder, mode='train')
val_dataset = SimpsonsDataset(val_files, label_encoder, mode='val')

batch_size = 64 #маленький батчсайз, Чтобы не пришлось шаманить в сохранением весов для избежания переполнения памяти

train_dataset[0][0].shape

torch.Size([3, 224, 224])
```

Реализация оверсемплинга, чтобы улучшить результат на малочисленных классах

```
from sklearn.utils.class_weight import compute_class_weight
from torch.utils.data import WeightedRandomSampler

train_labels_numeric = []

for i in range(len(train_dataset)):
    file_path = train_dataset.files[i]
    class_name = Path(file_path).parent.name
    label = train_dataset.label_encoder.transform([class_name]).item()
    train_labels_numeric.append(label)

class_weights = compute_class_weight('balanced', classes=np.unique(train_labels_numeric), y=train_labels_numeric)
class_weights = torch.tensor(class_weights, dtype=torch.float32).to(DEVICE)
sample_weights = class_weights[train_labels_numeric].cpu()
samp = WeightedRandomSampler( weights = sample_weights, num_samples = len(train_labels_numeric), replacement = True )

train_loader = DataLoader(train_dataset, batch_size=batch_size, sampler=samp, num_workers=4, pin_memory=True, prefetch_factor=3)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers= 4, pin_memory=True, prefetch_factor=3)
loaders = {'train':train_loader, 'val': val_loader}
```

Посмотрим на изображения персонажей

Напишем функцию для визуализации картинок. Это аналог Imshow, которая будет принимать на вход тензоры

```
def imshow(inp, title=None, plt_ax=plt, default=False):
    """Imshow для тензоров"""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    plt_ax.imshow(inp)
    if title is not None:
        plt_ax.set_title(title)
    plt_ax.grid(False)
```

Проверим, как работает наша функция. Возьмем один батч из лодера и подадим на вход функции первый тензор из этого батча. Заодно посмотрим, как работает кодировщик меток класса.

```
#image_tensor, label = next(iter(train_loader))
#print(f"Числовая метка класса: {label[0]}")
#print(f"Текстовая метка класса: {label_encoder.inverse_transform([label[0],])}")
#imshow(image_tensor[0])

# uncomment if you have problem with pillow
# def register_extension(id, extension): Image.EXTENSION[extension.lower()] = id.upper()
# Image.register_extension = register_extension
# def register_extensions(id, extensions):
#     for extension in extensions: register_extension(id, extension)
# Image.register_extensions = register_extensions
```

Давайте посмотрим на наших героев внутри датасета.

```
def show_images(n_rows, n_cols, dataset):
    # функция выводит изображения в n_rows рядов и n_cols столбцов
    fig, ax = plt.subplots(nrows=n_rows, ncols=n_cols, figsize=(n_cols * 4, n_rows * 4), \
                           sharey=True, sharex=True)

    for fig_x in ax.flatten(): # цикл повторяется (n_rows * n_cols) раз
        random_characters = int(np.random.uniform(0, len(dataset))) # берем случайное число в диапазоне [0; len(dataset))
        im_val, label = dataset[random_characters] # получаем изображение (тензорная форма)
        img_label = " ".join(map(lambda x: x.capitalize(), \
                                   label_encoder.inverse_transform([label])[0].split('_'))) # числовую метку класса преобразуем в текстовую и немного отформатируем
        imshow(im_val.data.cpu(), title=img_label, plt_ax=fig_x) # отобразим изображение и метку класса с помощью написанной ранее функции imshow()
        fig_x.set_axis_off()
    return None
```


! Обратите внимание, что возможно, датасет требует дополнительной фильтрации, так как лейблы не всегда точно соответствуют картинкам. Очистка датасета - это один из способов повысить качество модели. Этим можно будет заняться во второй части домашнего задания. Запустим написанную функцию. **

/

```
show_images(n_rows = 3, n_cols = 4, dataset = val_dataset)
```



Шаг 3. Построение нейросети

Запустить данную сеть будет вашим мини-заданием на первую неделю, чтобы было проще участвовать в соревновании.

Данная архитектура будет очень простой и нужна для того, чтобы установить базовое понимание и получить простенький сабмит на Kaggle.

О том, как работают свёртки подробнее можно прочитать в документации Pytorch на функцию `Conv2d`:

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Калькулятор свёрток:

<https://madebyollin.github.io/convnet-calculator/>

Модель

```
class SimpleCnn(nn.Module):
    """
    Очень простая сверточная нейронная сеть для классификации изображений.

    Эта сеть состоит из пяти сверточных слоев, каждый из которых
    включает в себя операцию свертки, функцию активации ReLU и операцию
    пулинга (max-pooling). На выходе используется полносвязный слой
    для классификации на заданное количество классов.

    Параметры:
    -----
    n_classes : int
        Количество классов для классификации.

    Примечание:
    -----
    Входные изображения должны иметь размерность (3, H, W), где
    3 - слои rgb для цветной картинки, а H и W - высота и ширина изображения,
    соответственно. Размер выходного тензора будет равен (n_classes).

    Методы:
    -----
    forward(x):
        Пропускает входные данные через сеть и возвращает logits для
        каждого класса.
    """

    def __init__(self, n_classes):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=8, kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Dropout2d(p=0.3)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels=8, out_channels=16, kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Dropout2d(p=0.3)
        )
        self.conv3 = nn.Sequential(
            nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3),
            nn.ReLU(),
```

```
nn.MaxPool2d(kernel_size=2),
nn.Dropout2d(p=0.3)
)
self.conv4 = nn.Sequential(
    nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Dropout2d(p=0.3)
)
self.conv5 = nn.Sequential(
    nn.Conv2d(in_channels=64, out_channels=96, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Dropout2d(p=0.3)
)

self.dropout = nn.Dropout(p=0.3)
self.out = nn.Linear(96 * 5 * 5, n_classes)

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    x = self.conv3(x)
    x = self.conv4(x)
    x = self.conv5(x)

    x = x.view(x.size(0), -1)
    x = self.dropout(x)
    logits = self.out(x)
    return logits
```

Напишите программный код или [сгенерируйте](#) его с помощью искусственного интеллекта.

Описание слоев:

- 1. размерность входа: $3 \times 224 \times 224$
- 2. размерность после 1-го слоя (Conv2d + ReLU + MaxPool2d): $8 \times 111 \times 111$
- 3. после 2-го слоя: $16 \times 54 \times 54$
- 4. после 3-го слоя: $32 \times 26 \times 26$
- 5. после 4-го слоя: $64 \times 12 \times 12$
- 6. после 5-го слоя: $96 \times 5 \times 5$
- 7. после полносвязного слоя (выход модели): количество классов

len(np.unique(train_val_labels))
42

Реализация архитектуры с skipconnections

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCnn2(nn.Module):
    def __init__(self, n_classes, dropout_rate=0.20):
        super().__init__()

        # Первый блок должен быть без residual connection
        self.conv_block1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(dropout_rate)
        )

        self.conv_block2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(dropout_rate)
        )

        # Shortcut для второго блока
        self.shortcut2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=1, stride=2),
            nn.BatchNorm2d(64)
        )

        self.conv_block3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(dropout_rate)
        )

        # Shortcut для третьего блока
        self.shortcut3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=1, stride=2),
            nn.BatchNorm2d(128)
        )

        self.conv_block4 = nn.Sequential(
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Dropout2d(dropout_rate)
        )

        # Shortcut для четвертого блока
```

```
self.shortcut4 = nn.Sequential(
    nn.Conv2d(128, 256, kernel_size=1, stride=2),
    nn.BatchNorm2d(256)
)

self.conv_block5 = nn.Sequential(
    nn.Conv2d(256, 512, kernel_size=3, padding=1),
    nn.BatchNorm2d(512),
    nn.ReLU(inplace=True),
    nn.Conv2d(512, 512, kernel_size=3, padding=1),
    nn.BatchNorm2d(512),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2),
    nn.Dropout2d(dropout_rate)
)

# Shortcut для пятого блока
self.shortcut5 = nn.Sequential(
    nn.Conv2d(256, 512, kernel_size=1, stride=2),
    nn.BatchNorm2d(512)
)

self.global_avg_pool = nn.AdaptiveAvgPool2d((1, 1))

self.classifier = nn.Sequential(
    nn.Dropout(dropout_rate),
    nn.Linear(512, 256),
    nn.ReLU(inplace=True),
    nn.Dropout(dropout_rate),
    nn.Linear(256, n_classes)
)

def forward(self, x):
    # Первый блок
    x = self.conv_block1(x)

    # Второй блок
    identity = x
    x = self.conv_block2(x)
    x = x + self.shortcut2(identity)
    x = F.relu(x)

    # Третий блок
    identity = x
    x = self.conv_block3(x)
    x = x + self.shortcut3(identity)
    x = F.relu(x)

    # Четвертый блок
    identity = x
    x = self.conv_block4(x)
    x = x + self.shortcut4(identity)
    x = F.relu(x)

    # Пятый блок
    identity = x
    x = self.conv_block5(x)
    x = x + self.shortcut5(identity)
    x = F.relu(x)

    #классификация, global avg нужен для усреднения каналов карт
    x = self.global_avg_pool(x)
    x = x.view(x.size(0), -1)
    x = self.classifier(x)

    return x
```

```
model_simple_cnn = SimpleCnn2(n_classes = len(np.unique(train_val_labels))).to(DEVICE)
```

```
torchsummary.summary(model_simple_cnn, (3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 224, 224]	896
BatchNorm2d-2	[-1, 32, 224, 224]	64
ReLU-3	[-1, 32, 224, 224]	0
Conv2d-4	[-1, 32, 224, 224]	9,248
BatchNorm2d-5	[-1, 32, 224, 224]	64
ReLU-6	[-1, 32, 224, 224]	0
MaxPool2d-7	[-1, 32, 112, 112]	0
Dropout2d-8	[-1, 32, 112, 112]	0
Conv2d-9	[-1, 64, 112, 112]	18,496
BatchNorm2d-10	[-1, 64, 112, 112]	128
ReLU-11	[-1, 64, 112, 112]	0
Conv2d-12	[-1, 64, 112, 112]	36,928
BatchNorm2d-13	[-1, 64, 112, 112]	128
ReLU-14	[-1, 64, 112, 112]	0
MaxPool2d-15	[-1, 64, 56, 56]	0
Dropout2d-16	[-1, 64, 56, 56]	0
Conv2d-17	[-1, 64, 56, 56]	2,112
BatchNorm2d-18	[-1, 64, 56, 56]	128
Conv2d-19	[-1, 128, 56, 56]	73,856
BatchNorm2d-20	[-1, 128, 56, 56]	256
ReLU-21	[-1, 128, 56, 56]	0
Conv2d-22	[-1, 128, 56, 56]	147,584
BatchNorm2d-23	[-1, 128, 56, 56]	256
ReLU-24	[-1, 128, 56, 56]	0
MaxPool2d-25	[-1, 128, 28, 28]	0
Dropout2d-26	[-1, 128, 28, 28]	0
Conv2d-27	[-1, 128, 28, 28]	8,320
BatchNorm2d-28	[-1, 128, 28, 28]	256
Conv2d-29	[-1, 256, 28, 28]	295,168
BatchNorm2d-30	[-1, 256, 28, 28]	512
ReLU-31	[-1, 256, 28, 28]	0
Conv2d-32	[-1, 256, 28, 28]	590,080
BatchNorm2d-33	[-1, 256, 28, 28]	512
ReLU-34	[-1, 256, 28, 28]	0
MaxPool2d-35	[-1, 256, 14, 14]	0
Dropout2d-36	[-1, 256, 14, 14]	0
Conv2d-37	[-1, 256, 14, 14]	33,024
BatchNorm2d-38	[-1, 256, 14, 14]	512
Conv2d-39	[-1, 512, 14, 14]	1,180,160
BatchNorm2d-40	[-1, 512, 14, 14]	1,024
ReLU-41	[-1, 512, 14, 14]	0
Conv2d-42	[-1, 512, 14, 14]	2,359,808
BatchNorm2d-43	[-1, 512, 14, 14]	1,024
ReLU-44	[-1, 512, 14, 14]	0
MaxPool2d-45	[-1, 512, 7, 7]	0
Dropout2d-46	[-1, 512, 7, 7]	0
Conv2d-47	[-1, 512, 7, 7]	131,584
BatchNorm2d-48	[-1, 512, 7, 7]	1,024
AdaptiveAvgPool2d-49	[-1, 512, 1, 1]	0
Dropout-50	[-1, 512]	0
Linear-51	[-1, 256]	131,328
ReLU-52	[-1, 256]	0
Dropout-53	[-1, 256]	0
Linear-54	[-1, 42]	10,794

Шаг 4. Функции для работы с моделью

Задание - напишите функции для работы с моделью

Обратите внимание, что метрика, которую необходимо оптимизировать в конкурсе -- `f1-score`.

Ниже приводится код для вычисления целевой метрики.

```
from sklearn.metrics import f1_score
# пример расчёта: f1 = f1_score(actual_labels, y_predicted, average='micro')
```

В этом задании может быть полезным сделать, чтобы функция валидации и, возможно, функция обучения на одной эпохе возвращали векторы/тензоры:

- предсказанного наиболее вероятного класса
- и фактических меток класса для каждого объекта.

Сама метрика `f1`, при этом, будет вычисляться не внутри функции, а вне ее, сопоставляя прогнозные и фактические значения, полученные из вывода функции.

Это поможет проанализировать, при предсказании каких классов модель ошибается чаще. Также такой подход может использоваться для обнаружения изображений, которые модель классифицирует неверно.

Для ускорения вычислений рекомендуется при работе с моделью проводить тензорные вычисления на GPU. Обращаем внимание, для этого необходимо, чтобы и данные, и модель находились на одном устройстве.

Ниже представлен код из предыдущего домашнего задания, который демонстрировал логику циклов обучения и валидации. Ниже в этот код добавлены строки комментариев, показывающие как переносить модель и данные на устройство.

```
model.to(DEVICE) # <- переносим модель на устройство DEVICE

for epoch in range(max_epochs):
    # режим train
    model.train()
    for x_batch, y_batch in train_dataloader:
        x_batch.to(DEVICE) # <- переносим тензоры данных на устройство DEVICE
        y_batch.to(DEVICE) # <- переносим тензоры данных на устройство DEVICE

        optimizer.zero_grad()
        outp = model(x_batch)
        loss = criterion(outp, y_batch)
        loss.backward()
        optimizer.step()

        count_metrics(outp, y_batch) # <-здесь, скорее всего, потребуется перенести данные на cpu, например с помощью метода cpu

    # режим test/eval
    model.eval()
    for x_batch, y_batch in val_dataloader:
        x_batch.to(DEVICE) # <- переносим тензоры данных на устройство DEVICE
        y_batch.to(DEVICE) # <- переносим тензоры данных на устройство DEVICE

        with torch.no_grad():
            outp = model(x_batch)
            count_metrics(outp, y_batch) # <-здесь, скорее всего, потребуется перенести данные на cpu
```

Напишите функции:

- для обучения модели на одной эпохе
- для валидации модели на одной эпохе
- для реализации полного цикла обучения

За основу можно взять функции, которые вы написали в предыдущем домашнем задании.

```
def fit_one_epoch(model, train_dataloader, optimizer, loss_func):
    """
    args:
    model - модель для обучения
    train_dataloader - loader с выборкой для обучения модели
    optimizer - оптимизатор, взятый из модуля `torch.optim`
    loss_func - функция потерь, взятая из модуля `torch.nn`

    функция возвращает метрику accuracy по эпохе на данных из train_dataloader
    """

    # YOUR CODE. переводим модель в режим train
    model.to(DEVICE)
    model.train()
    x = []
    y = []
    for X_batch, y_batch in tqdm(train_dataloader):
        # Далее далее по коду идут стандартные операции цикла обучения:
        # YOUR CODE. обнуляем градиенты модели
        X_batch = X_batch.to(DEVICE)
        y_batch = y_batch.to(DEVICE)
        optimizer.zero_grad()
        outp = model(X_batch) # YOUR CODE. Use model() to get outputs
        loss = loss_func(outp, y_batch)# YOUR CODE. Compute loss

        loss.backward()# YOUR CODE. считаем градиенты
        optimizer.step()# YOUR CODE. делаем шаг градиентного спуска

        # Рассчитаем accuracy. Для этого определим количество верных предсказаний
        # и общее количество предсказаний в каждом батче, а затем накопительным итогом посчитаем
        # количество верных предсказаний и общее количество предсказаний в рамках эпохи
        preds = outp.argmax(-1)
        x.extend(preds.detach().cpu().tolist())
        y.extend(y_batch.detach().cpu().tolist())
    return [x,y]
```

```
def eval_one_epoch(model, val_dataloader):
    """
    args:
    model - модель для обучения
    val_dataloader - loader с валидационной/тестовой выборкой
```



```
...
x = []
y = []
# YOUR CODE. Переводим модель в режим eval
model.to(DEVICE)
model.eval()
for X_batch, y_batch in tqdm(val_dataloader):
    X_batch = X_batch.to(DEVICE)
    y_batch = y_batch.to(DEVICE)
    # Далее далее по коду идут стандартные операции цикла валидации:
    # YOUR CODE. Не забываем отключить расчет градиентов
    with torch.no_grad():
        # YOUR CODE. Получаем "логиты" из модели
        logits = model(X_batch)
        preds = logits.argmax(-1)# YOUR CODE. Определяем нииболее вероятный класс для каждого объекта в батче

    x.extend(preds.detach().cpu().tolist())
    y.extend(y_batch.detach().cpu().tolist())
return [x,y]
```

```
from torch.optim.lr_scheduler import ReduceLROnPlateau, CosineAnnealingLR, StepLR
def train_func(model, num_epochs, dataloaders, optimizer, loss_func, scheduler):
    ...

    args:
    model - модель для обучения
    num_epochs - количество эпох
    dataloaders - словарь loader'ов с обучающей и валидационной выборками
    optimizer - оптимизатор, взятый из модуля `torch.optim`
    loss_func - функция потерь, взятая из модуля `torch.nn`

    функция возвращает ассигасу на обучающей и валидационной выборках на каждой эпохе
    ...

    f1 = {"train": [], "valid": []}
    tr = []
    val = []
    temp_f1 = 0
    for epoch in range(num_epochs):
        print(f"\nEpoch: {epoch+1}")

        train_one_epoch = fit_one_epoch(model = model, train_dataloader = dataloaders['train'], optimizer = optimizer, loss_func = loss_func)
        f1sc = f1_score(train_one_epoch[0], train_one_epoch[1], average='micro')

        if epoch >= num_epochs -1:
            tr.append([train_one_epoch])

        print(f"f1_train: {f1sc}\n")

        test_one_epoch = eval_one_epoch(model = model, val_dataloader = dataloaders['val'])
        f1scv = f1_score(test_one_epoch[0], test_one_epoch[1], average='micro')
        if epoch >= num_epochs -1:
            val.append([test_one_epoch])
        if temp_f1 < f1scv:
            temp_f1 = f1scv
            best_w = copy.deepcopy(model.state_dict())
            print('Сохранено!')

        scheduler.step(f1scv)
        new_lr = optimizer.param_groups[0]['lr']
        print(new_lr)
        print(f"f1_test: {f1scv}\n")
        f1['train'].append(f1sc)
        f1['valid'].append(f1scv)
    if temp_f1 != 0:
        model.load_state_dict(best_w)
    return [f1, tr, val]
```

▼ Шаг 5. Применение модели к данным

▼ Применение модели к данным

Запустите обучение сети

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model_simple_cnn.parameters(), lr = 3e-3, weight_decay = 3e-4)
scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=3)
f1 = train_func(model = model_simple_cnn, num_epochs = 20, dataloaders = loaders,optimizer = optimizer, loss_func= criterion, scheduler = scheduler)
#Обучение слетело, но модель показала точность 96% за 40 эпох и могла обучаться дальше, заново обучил только на 20
#чтобы не терять время, т.к. предобученная модель показала результат гораздо лучше за меньшее время
```

```
Epoch: 1
100%|██████████| 246/246 [01:38<00:00, 2.50it/s]
f1_train: 0.08013249251544684

100%|██████████| 82/82 [00:12<00:00, 6.83it/s]
Сохранено!
0.003
f1_test: 0.04088651127244937

Epoch: 2
100%|██████████| 246/246 [01:50<00:00, 2.22it/s]
f1_train: 0.15720746544365882

100%|██████████| 82/82 [00:12<00:00, 6.56it/s]
Сохранено!
0.003
f1_test: 0.08215513947267863

Epoch: 3
100%|██████████| 246/246 [01:51<00:00, 2.21it/s]
f1_train: 0.22778520924899676

100%|██████████| 82/82 [00:12<00:00, 6.72it/s]
Сохранено!
0.003
f1_test: 0.13412304165074512

Epoch: 4
100%|██████████| 246/246 [01:51<00:00, 2.20it/s]
f1_train: 0.29027326581310914

100%|██████████| 82/82 [00:12<00:00, 6.60it/s]
Сохранено!
0.003
f1_test: 0.20787160871226595
```

```
Epoch: 5
100%|██████████| 246/246 [01:51<00:00, 2.20it/s]
f1_train: 0.3799605070386649

100%|██████████| 82/82 [00:12<00:00, 6.60it/s]
Сохранено!
0.003
f1_test: 0.3354986625907528

Epoch: 6
100%|██████████| 246/246 [01:51<00:00, 2.21it/s]
f1_train: 0.4560163067711319

100%|██████████| 82/82 [00:12<00:00, 6.63it/s]
Сохранено!
```

Дообучение efficientnet

Данных много, поэтому будем тренировать модель полностью

```
import torch
import torch.nn as nn
import torchvision.models as models
from torchvision.models import EfficientNet_B3_Weights

# 1. Загрузка
weights = EfficientNet_B3_Weights.DEFAULT
model = models.efficientnet_b3(weights=weights)

Downloading: "https://download.pytorch.org/models/efficientnet_b3_rwightman-b3899882.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b3_rwightman-b3899882.pth
100%|██████████| 47.2M/47.2M [00:00<00:00, 181MB/s]

print(model)

Показать скрытые выходные данные
```

```
#требуется изменить только классификатор
n_classes = len(np.unique(train_val_labels))
in_fts = model.classifier[1].in_features

#модель училась на imagenet с большим количеством классов, поэтому решил чуть-чуть сократить, Чтобы не прыгать сразу от 1500 параметров к 3

model.classifier = nn.Sequential(
    nn.Dropout(0.2, inplace=True),
    nn.Linear(in_fts, in_fts//2),
    nn.Dropout(0.2, inplace=True),
    nn.Linear(in_fts//2, in_fts//2),
    nn.Dropout(0.2, inplace=True),
    nn.Linear(in_fts//2, n_classes),
)

model = model.to(DEVICE)
#можно переделать преобразования картинок под модель (есть встроенный список преобразований),
#но я не стал, потому что модель уже на первой эпохе показала результат в 0.94
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5, weight_decay=3e-4) # поставил маленький lr
criterion = nn.CrossEntropyLoss()
scheduler = ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=3)

effout = train_func(model, 15, loaders, optimizer, criterion, scheduler)
#стоило бы усилить аугментацию, и переобучить, но ломалась бы написанная ранее модель,
#потому что не потянула бы слишком сильной аугментации, пришлось бы гораздо дольше учить
```

```
Epoch: 1
100%|██████████| 246/246 [03:00<00:00, 1.36it/s]
f1_train: 0.3209758583349258

100%|██████████| 82/82 [00:15<00:00, 5.26it/s]
Сохранено!
3e-05
f1_test: 0.5032479938861292

Epoch: 2
100%|██████████| 246/246 [02:59<00:00, 1.37it/s]
f1_train: 0.8054653162621823

100%|██████████| 82/82 [00:15<00:00, 5.25it/s]
Сохранено!
3e-05
f1_test: 0.8926251432938479

Epoch: 3
100%|██████████| 246/246 [02:59<00:00, 1.37it/s]
f1_train: 0.9345181221733868

100%|██████████| 82/82 [00:15<00:00, 5.27it/s]
Сохранено!
3e-05
f1_test: 0.92338555598013

Epoch: 4
100%|██████████| 246/246 [02:59<00:00, 1.37it/s]
f1_train: 0.9596152621186063

100%|██████████| 82/82 [00:15<00:00, 5.24it/s]
Сохранено!
3e-05
f1_test: 0.938479174627436

Epoch: 5
100%|██████████| 246/246 [02:59<00:00, 1.37it/s]
f1_train: 0.9718453404675457

100%|██████████| 82/82 [00:15<00:00, 5.28it/s]
Сохранено!
3e-05
f1_test: 0.9476499808941536

Epoch: 6
100%|██████████| 246/246 [02:59<00:00, 1.37it/s]
f1_train: 0.9787247595388241
```

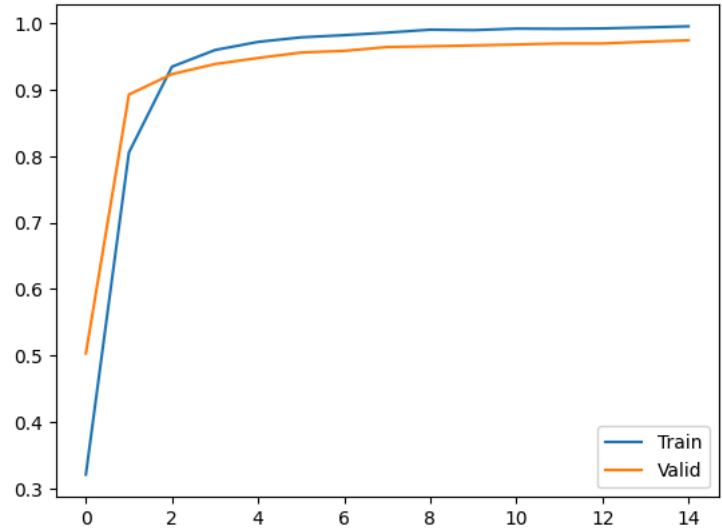
На 15 эпохе модель получила метрику f1 = 0.974

```
torch.save(model.state_dict(), 'model_weights.pth')
torch.save(model, 'full_model.pth')
```

(опционально) Постройте график динамики метрик модели по ходу обучения

```
import matplotlib.pyplot as plt

plt.plot(effout[0]['train'], label='Train')
plt.plot(effout[0]['valid'], label='Valid')
plt.legend()
plt.show()
```



```
with torch.no_grad():
    test_one_epoch = eval_one_epoch(model = model, val_dataloader = loaders['val'])

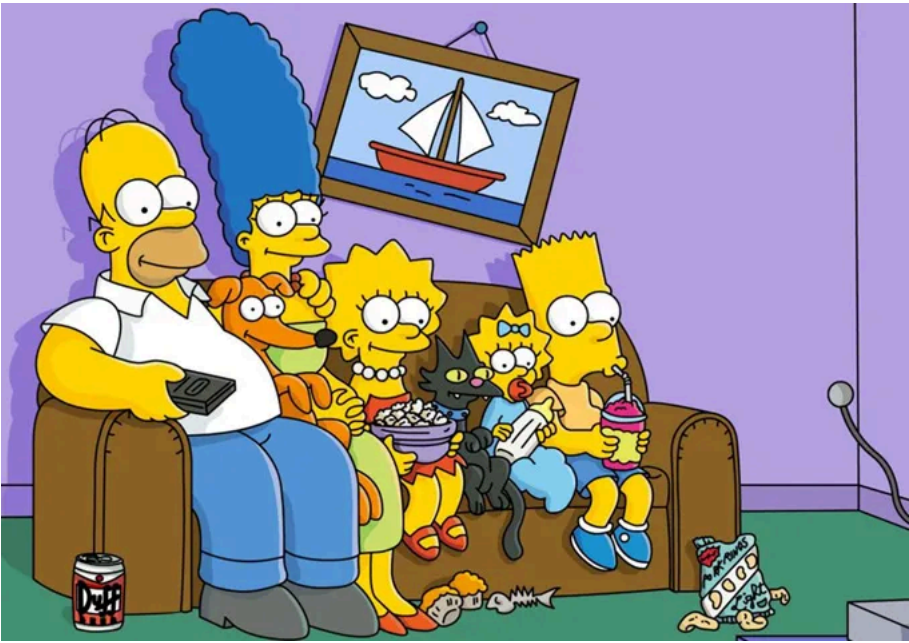
f1scvv = f1_score(test_one_epoch[0], test_one_epoch[1], average='micro')
```

```
f1scvv

0.9742071073748567
```

Итоговый f1 score: 0.974

Ну и что теперь со всем этим делать?



Сделаем классную визуализацию, чтобы посмотреть насколько сеть уверена в своих ответах.

```
import matplotlib.patches as patches
from matplotlib.font_manager import FontProperties

@torch.no_grad()
def show_images2(n_rows, n_cols, dataset, model):
    # функция выводит изображения в n_rows рядов и n_cols столбцов
    fig, axs = plt.subplots(nrows=n_rows, ncols=n_cols, figsize=(n_cols * 4, n_rows * 4), \
                           sharey=True, sharex=True)

    for fig_x in axs.flatten(): # цикл повторяется (n_rows * n_cols) раз
        random_characters = int(np.random.uniform(0, len(dataset))) # берем случайное число в диапазоне [0; len(dataset))
        im_val, label = dataset[random_characters] # выводим изображение (тензорная форма)
        img_label = " ".join(map(lambda x: x.capitalize(), \
                                  label_encoder.inverse_transform([label])[0].split('_'))) # числовую метку класса преобразуем в текстовую и немного отформатируем
        imshow(im_val.data.cpu(), title=img_label, plt_ax=fig_x) # отобразим изображение и метку класса с помощью написанной ранее функции imshow()

    # ниже представлен код для добавления на изображение подписи с вероятностью класса
    actual_text = "Actual : {}".format(img_label)

    font0 = FontProperties()
    font = font0.copy()

    # наша модель возвращает логиты, поэтому для получения вероятностей классов не забываем применить softmax:
    prob_pred = nn.functional.softmax(model(im_val.unsqueeze(0).to(DEVICE)), dim = -1).cpu().numpy()

    predicted_proba = np.max(prob_pred)*100 # берем максимальную вероятность класса
```

```
y_pred = np.argmax(prob_pred) # берем числовую метку класса, для которого вероятность максимальна

predicted_label = " ".join(map(lambda x: x.capitalize(), \
                               label_encoder.inverse_transform([y_pred])[0].split('_')))
predicted_text = "{}:\n {:.1f}%".format(predicted_label, predicted_proba)

fig_x.add_patch(patches.Rectangle((0, 190), 7*len(predicted_label), 25,color='white'))
fig_x.text(2, 195, predicted_text , horizontalalignment='left', fontproperties=font,
          verticalalignment='top',fontsize=8, color='black',fontweight='bold')
fig_x.set_axis_off()

return None
```



Шаг 6. Submit на Kaggle



Создадим loader для тестовых данных

```
test_dataset = SimpsonsDataset(test_files, label_encoder = label_encoder, mode="test")
test_loader = DataLoader(test_dataset, shuffle=False, batch_size=64)
```

Воспользуемся функцией predict, которая возвращает предсказанные числовые метки для всех объектов в лодере.


```
def predict(model, loader):
    model.eval()
    all_predictions = torch.tensor([]).to(DEVICE).int()
    print("Test mode...")
    for inputs in tqdm_notebook(loader):
        inputs = inputs.to(DEVICE)

        with torch.no_grad():
            outputs = model(inputs)

            predictions = outputs.argmax(-1).int()
            all_predictions = torch.cat((all_predictions, predictions), 0)
    return all_predictions.cpu()
```

Получем предсказание меток классов для тестовых данных:

```
predicted_numeric_labels = predict(model, test_loader)

Test mode...
0%|          | 0/16 [00:00<?, ?it/s]
```

и преобразуем их в текстовые метки:

```
predicted_text_labels = label_encoder.inverse_transform(predicted_numeric_labels)
```

Загрузим пример файла для загрузки на Kaggle (проверьте путь, по которому у вас лежит файл sample_submission.csv и при необходимости скорректируйте путь в коде ниже):

```
import pandas as pd
sample_submission = pd.read_csv("/kaggle/working/sample_submission.csv")
sample_submission.head(10)
```

	Id	Expected
0	img0.jpg	bart_simpson
1	img1.jpg	bart_simpson
2	img2.jpg	bart_simpson
3	img3.jpg	bart_simpson
4	img4.jpg	bart_simpson
5	img5.jpg	bart_simpson
6	img6.jpg	bart_simpson
7	img7.jpg	bart_simpson
8	img8.jpg	bart_simpson
9	img9.jpg	bart_simpson

```
my_submission = pd.DataFrame({'Id': [path.name for path in test_files], 'Expected': predicted_text_labels})
my_submission.head(10)
```

	Id	Expected
0	img0.jpg	nelson_muntz
1	img1.jpg	bart_simpson
2	img10.jpg	ned_flanders
3	img100.jpg	chief_wiggum
4	img101.jpg	apu_nahasapeemapetilon
5	img102.jpg	kent_brockman
6	img103.jpg	edna_krabappel
7	img104.jpg	chief_wiggum
8	img105.jpg	lisa_simpson
9	img106.jpg	kent_brockman

```
# TODO : сделайте сабмит (это важно, если Вы не справляетесь, но дошли до этой ячейки, то сообщите в чат и Вам помогут)
```

```
my_submission.to_csv('cnnanswww.csv', index=False)
```

Приключение?

А теперь самое интересное, мы сделали простенькую сверточную сеть и смогли отправить сабмит, но получившийся score нас явно не устраивает. Надо с этим что-то сделать.

Вам предлагается творческая и исследовательская свобода в том, как вы будете улучшать score модели. Вот некоторые направления, которые вы, возможно, захотите попробовать:

1. Работа с датасетом:
 - Попробуйте увеличить разнообразие обучающей выборки при помощи аугментации ([ссылка 1](#), [ссылка 2](#));
 - Используя результаты функции eval_one_epoch, вы можете посмотреть, как метрика f1-score внутри класса связана с размером самого класса в обучающей выборке. Скорее всего, вы заметите прямую зависимость. Решением может быть увеличение размеров слабо представленных классов. Сделать это можно, вручную добавив этот функционал в SimpsonsDataset или воспользовавшись инструментом WeightedRandomSampler.
 - Также вы можете изучить, какие изображения модель классифицирует неверно. Можно просто посмотреть неверно классифицированные изображения в валидационном датасете. Можно попробовать более фундаментальный подход - создать dataloader из всех изображений обучающей и валидационной выборки в SimpsonsDataset с режимом 'val', прогнать данные через обученную модель simple_cnn. Более того, можно обучить несколько моделей с разной архитектурой, и прогнать объединенный датасет через все эти модели - и выделить те изображения, в классификации которых ошиблись несколько моделей.
 - Это поможет понять, как качество и чистота исходных данных влияет на результаты обучения модели. Если вы обнаружите ошибки в simpsons_dataset и решите их исправить или удалить некоторые изображения из обучающей выборки - сделайте это в ноутбук, чтобы при проверке можно было повторить результаты работы вашего кода.
2. Работа с архитектурой сети:
 - учим дольше и изменяем (подбираем) гиперпараметры: learning rate, batch size, нормализация картинки и вот это всё;
 - добавление/удаление свёрточных и полносвязных слоёв;
 - А еще есть где пулинги и батч-норм!
3. Применение техник, описанных в предыдущих пунктах, позволяет добиться довольно высоких результатов. Но вы можете попробовать файнтюнинг и трансферлёрнинг:

- ну разве это deep learning? Вот ResNet и Inception, которые можно зафайнтюнить под наши данные, вот это я понимаю (можно и обучить в колабе, а можно и [готовые](#) скачать).

4. Улучшение оптимизации:

- ну разве Адам наше все? [adamW](#) для практика, [статейка для любителей](#) (очень хороший анализ), [наши](#) эксперименты для заинтересованных.

5. Наконец, возможно, стоит подумать об ансамблях

Надеюсь, что у Вас получится!

✖ Итоговое качество:

в ноутбуке: f1 = 0.974

на kaggle: f1 = 0.993

f1scvv
0.9742071073748567